Relazione Progetto Reti Logiche
A.S. 2024/2025

Davide Ponzo, 10842752
Filippo Romano, 10855364

Contents

1 Introduzione e descrizione funzionale **3**

2 Specifica funzionale **3**
2.1 Esempio 3

3 Architettura **5**
3.1 Interfaccia VHDL 5
3.2 Architettura della macchina a stati 6
3.3 Descrizione dei processi 7

4 Risultati sperimentali **7**
4.1 Risultati di sintesi 8
4.1.1 Utilizzo di risorse fisiche 8
4.1.2 Slack e timing 8
4.2 Risultati di test 8

5 Conclusioni **8**

1 Introduzione e descrizione funzionale

In questa relazione si documenta il processo di realizzazione e test di un componente hardware, simulato in linguaggio VHDL. L'obiettivo del modulo è la lettura, filtraggio e scrittura di sequenze numeriche. Il filtro può essere di due ordini diversi, e consiste nella modifca della serie iniziale attraverso dei coefficienti ricevuti da memoria. Questa operazione verrà discussa nel dettaglio nella prossima sezione del documento. Il componente si interfaccia con una memoria di 2^{16} elementi, con indirizzi quindi espressi in 16 bit. Ogni elemento è a sua volta lungo 8 bit.

2 Specifica funzionale

Il componente possiede, secondo la specifica, le seguenti funzionalità. Salvo specificato, tutte le funzionalità avvengono in modo sincrono al clock del sistema.

- Lettura e salvataggio in memoria di una sequenza di setup di 17 byte, letta prima della sequenza da filtrare, a partire da un indirizzo di memoria ricevuto in ingresso tramite un segnale input **i_addr**.
  La sequenza contiene, in ordine:
  - La lunghezza K della sequenza da analizzare, indicata in 2 byte, K1 e K2;
  - L'ordine del filtro da applicare, indicato dal bit meno significativo di un byte: bit = 0 per il filtro di ordine 3, bit = 1 per il filtro di ordine 5;
  - 14 bit contenenti i coefficienti utilizzati da entrambi i filtri, 7 per ogni filtro.
- Operazione di filtraggio su ogni elemento della sequenza di lunghezza K, diviso nei seguenti passaggi:
  - Lettura dell'elemento di 8 bit da filtrare;
  - Lettura e salvataggio temporaneo in memoria dei valori adiacenti di 2 o 3 posizioni, rispettivamente nel filtro di ordine 3 o di ordine 5;
  - Aggiornamento del valore del primo elemento secondo la seguente formula:

    $$f'[i] = \frac{1}{n} * \Sigma C_j * f[i+j]$$

    Dove f è la sequenza originale, f' è la sequenza filtrata, j è un valore che oscilla tra -2 e +2 nel caso di filtro di ordine tre, e tra -3 e +3 per il filtro di ordine 5; n è un valore di normalizzazione, pari a 12 nel filtro di ordine 3, e 60 in quello di ordine 5. Infine, C_j è il valore dei coefficienti, ricevuti durante la fase di setup. Nel caso particolare in cui i valori adiacenti non siano parte della sequenza f, vengono considerati pari a 0.
  - Scrittura in memoria del valore filtrato, in sequenza, a partire dall'indirizzo successivo alla sequenza K da leggere. Prima della scrittura, il valore viene saturato tra i valori -128 e +127.
- Reset asincrono del componente, in seguito alla ricezione di un segnale **i_rst**. Durante il reset, il componente azzera tutti i suoi parametri, e si mette in attesa del prossimo segnale di inizio.

2.1 Esempio

Qui di seguito un esempio banale del funzionamento del filtro:
Sequenza iniziale: 1 2 3 4 3 2 1

Coefficienti del filtro di ordine 2: 0 -1 3 2 4 -3 0
Applicando il filtro di ordine 2 al quarto elemento della sequenza si ha:

$$f'[3] = \frac{1}{n} * \Sigma C_j * f[3+j] =$$

$$= \frac{1}{n} * [(2 * -1) + (3 * 3) + (4 * 2) + (3 * 4) + (1 * -3)] = \frac{1}{n} * 21$$

Per la divisione, ragioniamo in binario: 21 -¿ 010101 In realtà, il numero andrebbe esteso a 16 bit, essendo il prodotto di due numeri codificati a 8 bit. Qui per ordine è espresso solo il numero minimo di bit necessari. Applicando gli shift a destra necessari, che per il filtro di ordine 3 sono di 4, 6, 8 e 10, si ottiene
1+0+0+0=1.
Il filtro realizzato offre quindi una divisione approssimativa, a causa del troncamento prodotto dallo shift. Questo comportamento è previsto dalla specifica.

3 Architettura

3.1 Interfaccia VHDL

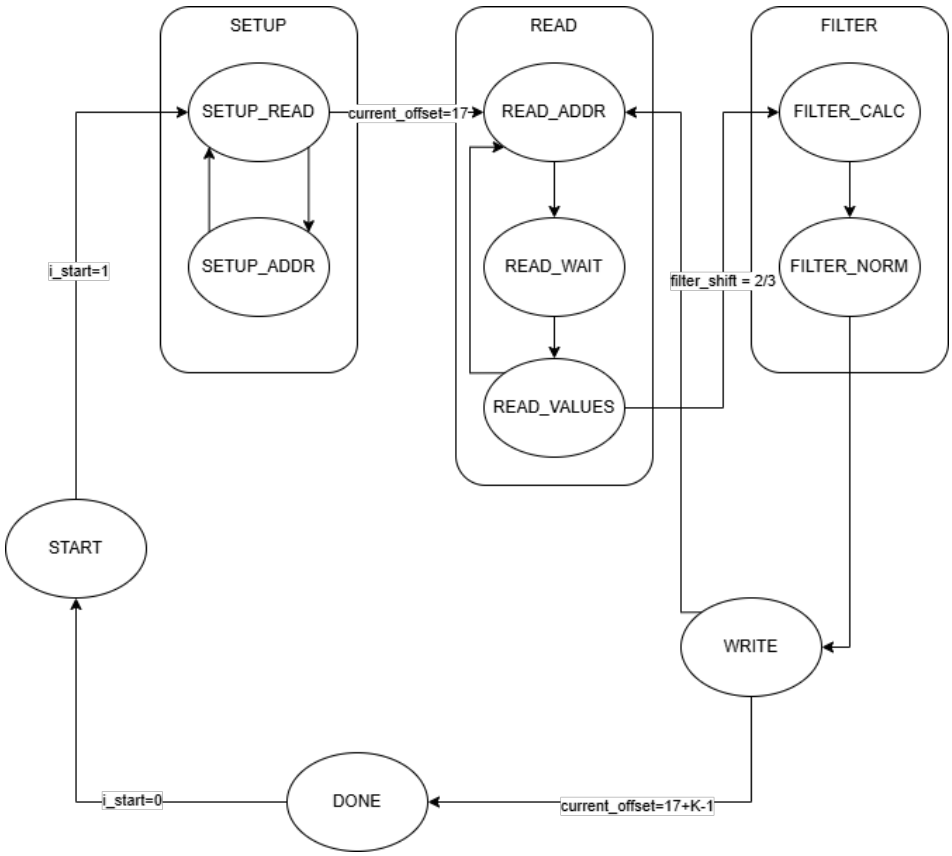
Qui di seguito è l'interfaccia VHDL del componente, come data dalla specifica, e il significato dei segnali.

```
entity project_reti_logiche is
    port (
        i_clk   : in std_logic;
        i_rst   : in std_logic;
        i_start : in std_logic;
        i_add   : in std_logic_vector(15 downto 0);

        o_done  : out std_logic;

        o_mem_addr : out std_logic_vector(15 downto 0);
        i_mem_data : in std_logic_vector(7 downto 0);
        o_mem_data : out std_logic_vector(7 downto 0);
        o_mem_we   : out std_logic;
        o_mem_en   : out std_logic
    );
end project_reti_logiche;
```

- **i_clk**: Segnale binario di clock ricevuto in ingresso, generato da test bench. Ogni funzione del componente avviene sul rising edge del ciclo di clock.
- **i_rst**: Segnale binario per il reset asincrono del componente. Se ricevuto, il componente azzera ogni valore salvato in memoria, e si mette in attesa di un nuovo segnale di inizio.
- **i_start**: Segnale binario che indica l'inizio del processo. Quando è ricevuto, il componente si mette in stato di lettura della sequenza di ingresso.
- **i_add**: Segnale a 16 bit, indica l'indirizzo di memoria da cui iniziare la lettura sequenziale.
- **o_done**: Segnale binario usato per segnalare la conclusione del processo per una data sequenza. Viene impostato a 1 quando il processo è terminato. Il componente rimane in uno stato "DONE" fino all'abbassamento del segnale di start, o fino al segnale di reset.
- **o_mem_addr**: Segnale a 16 bit mandato in uscita, comunica l'indirizzo di memoria su cui far avvenire le operazioni di lettura o scrittura.
- **i_mem_data**: Segnale a 8 bit ricevuto in ingresso, rappresenta un elemento letto da memoria.
- **o_mem_data**: Segnale a 8 bit mandato in uscita, rappresenta un elemento da scrivere in memoria.
- **o_mem_we**: Segnale binario di write enable. Se posto a 0, comunica alla memoria di leggere valori, se posto a 1, di scrivere.
- **o_mem_en**: Segnale binario di enable, deve essere posto a 1 per permettere operazioni di read/write in memoria.

3.2 Architettura della macchina a stati

Il componente è implementato come FSM. Gli stati possono essere divisi nei seguenti gruppi: START, SETUP, READ, FILTER, WRITE, DONE. La logica del passaggio da un gruppo ad un altro non è sempre sequenziale, ma anche è basata sulla ricezione di segnali in ingresso da memoria, o dal conteggio del numero di elementi letti, salvato nel segnale **current_offset**.



Si noti che nel grafico è stata omessa per questione di ordine la transizione, sempre possibile, da uno stato qualsiasi a **START** nel caso della ricezione del segnale **i_rst**.

- **START**: Lo stato di stand-by del componente. In questo stato stato i parametri non sono inizializzati, e il componente attende il segnale **i_start** per passare alla fase di setup.
- **SETUP**:
  - **SETUP_READ**: Stato principale della fase di setup, in cui i dati della sequenza iniziale di 17 caratteri vengono salvati in memoria, e altri parametri quali la lunghezza K della sequenza e l'ordine del filtro vengono inizializzati.
  - **SETUP_ADDR**: Stato ausiliario che assicura il corretto allineamento tra gli indirizzi della memoria esterna e quella interna del componente, e incrementa il valore di **current_offset** condizionalmente.
- **READ**: Nella fase di lettura vengono letti sia il valore da filtrare, sia quelli adiacenti a seconda dell'ordine del filtro.
  - **READ_ADDR**: Stato ausiliario che si occupa dell'incremento di **current_offset**.

- **READ_WAIT**: Stato ausiliario, utilizzato solo per assicurare il corretto allineamento tra memoria interna ed esterna.
- **READ_VALUES**: Stato principale della fase di lettura. Contiene la lettura in memoria e la logica di incremento di un segnale **filter_shift** per la lettura dei valori adiacenti.

- **FILTER**: Gruppo di stati che si occupa delle operazioni matematiche del filtro. Il calcolo è stato diviso in due stati per accertarsi che la durata di ogni fase fosse minore di un ciclo di clock.
  - **FILTER_CALC**: Si occupa di calcolare $\Sigma C_j * f[i+j]$, cioè la sommatoria degli elementi letti dalla sequenza in memoria, moltiplicati per i coefficienti del filtro ricevuti in fase di setup.
  - **FILTER_NORM**: Si occupa della normalizzazione del valore ricavato in **FILTER_CALC**. Lavorando in logica binaria, la divisione per 12 e 60 è gestita come la somma di divisioni per fattori di 2, ovvero l'operazione di shift offerta da VHDL.
- **WRITE**: Stato semplice, scrive in memoria l'ultimo valore filtrato, occupandosi anche della saturazione tra -128 e +127.
- **DONE**: Stato conclusivo. Si occupa solo di mandare il segnale **o_dove**, finchè il segnale **i_start** non torna a 0, segnalando la fine del processo.

3.3 Descrizione dei processi

Dal punto di vista implementativo in VHDL, il componente è realizzato con un architettura di tipo behavioral, divisa in tre processi concorrenti.

- **Processo di logica degli stati**
  Un processo si occupa della logica del passaggio tra stati. Attraverso un costrutto **case**, il processo sceglie lo stato che verrà raggiunto al ciclo di clock successivo in base allo stato attuale, e lo manda su un segnale **next_state**. In alcuni casi il passaggio è immediato (es: dallo stato di normalizzazione "FILTER_NORM" si passa sempre allo stato "WRITE"), in altri è basato su condizioni specifiche (es: il componente rimane in stato "START" fino alla ricezione di un segnale specifico). Questo processo ha come sensibilità il segnale **i_clk**.
- **Processo di cambio di stato e reset**
  Un processo si occupa del cambio di stato ad ogni ciclo di clock, in base alle decisioni prese dal processo sopra descritto. Questo è anche il processo che si occupa del reset asincrono del modulo. La sensibilità di questo processo, infatti, è sia **i_clk**, sia **i_rst**.
- **Processo principale**: Questo processo si occupa di tutte le funzioni principali del componente. Usando un costrutto **case**, questo processo compie le azioni di lettura, salvataggio, filtraggio e scrittura a seconda dello stato attuale. Essendo tutte queste funzioni sincrone al clock, la sensibilità è **i_clk**.

4 Risultati sperimentali

In questa sezione sono discussi i dati ottenuti dalla simulazione post-sintesi del componente, e i test utilizzati per accertarsi del corretto funzionamento dell'implementazione realizzata.

4.1 Risultati di sintesi

4.1.1 Utilizzo di risorse fisiche

Qui di seguito è riportato l'utilizzo delle risorse fisiche, calcolato su un dispositivo della famiglia Artix-7.

- **LUT**: L'implementazione utilizza il **6,46%** delle look up table disponibili.
- **FF**: Il numero di Flip_Flop utilizzati è lo **0,71%** di quelli disponibili.
- **IOB**: Il **50,94%** delle risorse di input/output è utilizzato.
- **Clock**: Il componente utilizza un singolo buffer globale (BUFG) per la gestione del segnale di clock.

Questi dati dimostrano che l'implementazione fa un uso moderato delle risorse disponibili, mantenendo l'utilizzo di LUT e FF ad un livello basso. L'alto bisogno di risorse di IO è giustificato, dato l'alto numero di segnale in input e output gestiti dal componente, anche di multipli bit.

4.1.2 Slack e timing

Lo strumento di sintesi di Vivado ha riportato i seguenti dati sullo slack del componente:

Worst negative slack(WNS)	7,948ns
Total negative slack(TNS)	0ns
Worst hold slack(WHS)	0,077ns
Total hold slack(THS)	0ns
Worst pulse width slack(WPWS)	9,500ns
Total pulse width slack(TPWS)	0ns
Number of failing endpoints	0

Considerando un periodo di clock dato da specifica di 20ns, questi valori dimostrano un'implementazione stabile, dove le restrizioni di timing non vengono mai violate, e con un margine considerevole.

4.2 Risultati di test

Nel verificare il corretto funzionamento del componente, sono stati utilizzati due testbench:

- **Test base**: Il test bench fornito dalla specifica, necessario per verificare il funzionamento accettabile del componente. Tutti i risultati di sintesi riportati sono stati registrati utilizzando questo test.
- **Test completo**: Un test bench di grandi dimensioni, che ha permesso di verificare il corretto funzionamento del componente anche in casi limite e di maggiore utilizzo di memoria (circa 48KB).

5 Conclusioni

In conclusione, il componente VHDL realizzato rispetta correttamente tutte le specifiche date. La macchina a stati realizzata, seppur abbia margine di miglioramento, e consistente e affidabile, come dimostrato anche da test più esigenti. I risultati statistici dimostrano inoltre che l'architettura realizzata permette di avere un discreto margine sul timing, con una frequenza massima possibile di circa 83MHz, contro i 50MHz richiesti dalla specifica. Infine, il componente dimostra un utilizzo delle risorse ridotto, dimostrando che la logica implementativa è sufficientemente snella.